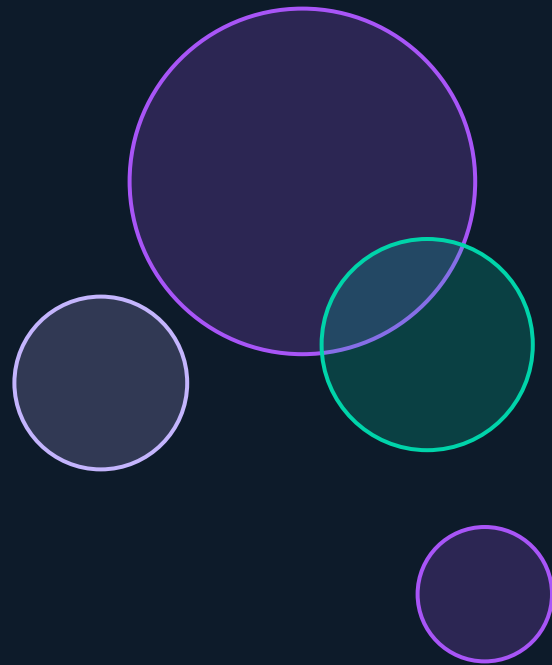


Recommender Systems

From Collaborative Filtering to Deep Learning

Dr. Nehal Ali & Dr. Moataz Samy

Week 10 • CS313



Agenda

01

What is a Recommender System?

02

Building User Profiles

03

Content-Based Filtering

04

Collaborative Filtering

05

Matrix Factorization

06

Deep Learning Models

07

Production Pipeline

08

Evaluation Metrics

09

Challenges, Bias & Fairness

10

Industry Case Studies

What is a Recommender System?



Search

PULL · Explicit Intent

User knows what they want.
"I want X."
System responds to a query.



RecSys

PUSH · Implicit Intent

User doesn't know it exists.
"You might like Y."
System proactively discovers.



Long Tail

The Business Case

Digital stores can surface niche items
(the bottom 80%) that physical stores
ignore.



"Recommendation is just Search with the user's context as the query."

Building User Profiles: Feedback Types

The raw material of every recommender — how we know what a user likes.

EXPLICIT FEEDBACK

"I tell you what I like."

★ Ratings (1–5 stars)

👍 Likes / Thumbs Up

📝 Written Reviews

Volume: LOW · Precision: HIGH · Bias:
HIGH

IMPLICIT FEEDBACK

"My actions show what I like."

🖱️ Clicks & Page Views

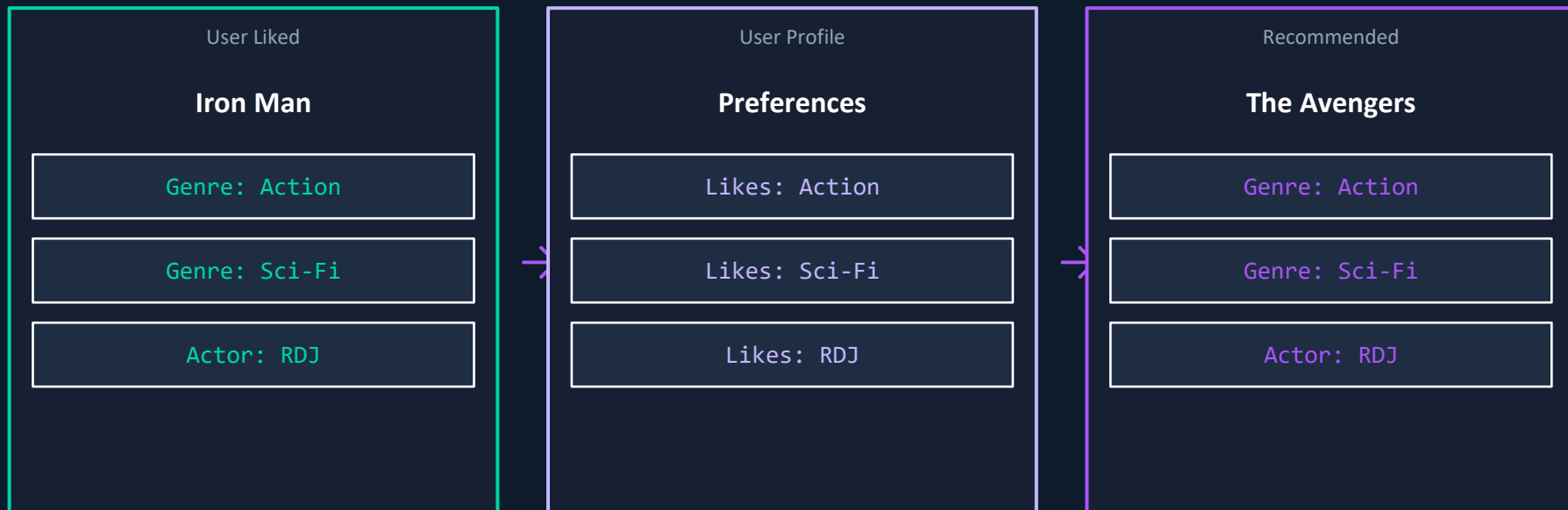
🛒 Purchases & Add-to-Cart

🕒 Watch Time / Dwell Time

Volume: MASSIVE · Precision: NOISY ·
Bias: LOW

Content-Based Filtering

Recommend items similar to what a user liked before, using item metadata.



✓ Advantages

No item cold start · User independence · Transparent & explainable

✗ Limitations

Filter bubble (no serendipity) · Feature engineering heavy · New user cold start

Collaborative Filtering (CF)

Leveraging the Wisdom of the Crowd — no item features needed, only interaction patterns.

User-User CF

Find the K most similar users (neighbors) using cosine/Pearson similarity. Predict rating as weighted average of neighbor ratings.

Item-Item CF (Amazon, 2003)

Pre-compute item-item similarity from co-occurrence. Items are stable; users change. "Customers who bought X also bought Y."

Key Insight

CF doesn't care what items ARE — only HOW users interact with them. Works for movies, books, socks, jokes.



Advantages

- Serendipity — discovers unexpected interests
- Content agnostic — domain independent
- Gets better as more users join



Limitations

- Cold start — new users and new items
- Data sparsity — most ratings are missing
- Popularity bias — recommends blockbusters

Matrix Factorization

Decompose the sparse User-Item matrix into compact Latent Factor (Embedding) matrices.



k Latent Factors (e.g. "Action", "Drama") — learned automatically from ratings

SVD / SGD

Iterative updates per rating. Easy to add bias terms. Netflix Prize winner (2009). Limited parallelism.

ALS (Alternating Least Squares)

Fix one matrix, solve for the other. Massively parallelizable (Apache Spark). Best for implicit feedback.

Visualizing Latent Factors

The model discovers hidden dimensions automatically — we never label them manually.

Latent Factors

Hidden dimensions the model discovers: e.g. Action vs. Romance, Serious vs. Funny. Never manually defined.

Dot Product = Rating

Rating $\approx$ User Vector $\cdot$ Item Vector.
Small angle between vectors = high similarity = high predicted rating.

SVD (Eckart-Young, 1936)

Keep only the top-k singular values to get the best low-rank approximation — filters noise, fills missing values.



Deep Learning for RecSys

Neural CF (NCF, 2017)

Replaces the fixed dot product of MF with a learnable MLP.
Learns non-linear user-item interactions.

Wide & Deep (Google Play, 2016)

Wide = memorization (logistic regression on co-occurrences).
Deep = generalization (DNN embeddings).

Two-Tower (YouTube)

Separate Query Tower (user) and Candidate Tower (item). Item vectors stored in FAISS for fast ANN search.

DeepFM

FM component (low-order feature interactions) + Deep component (high-order interactions). Best for CTR.

BERT4Rec / GRU4Rec

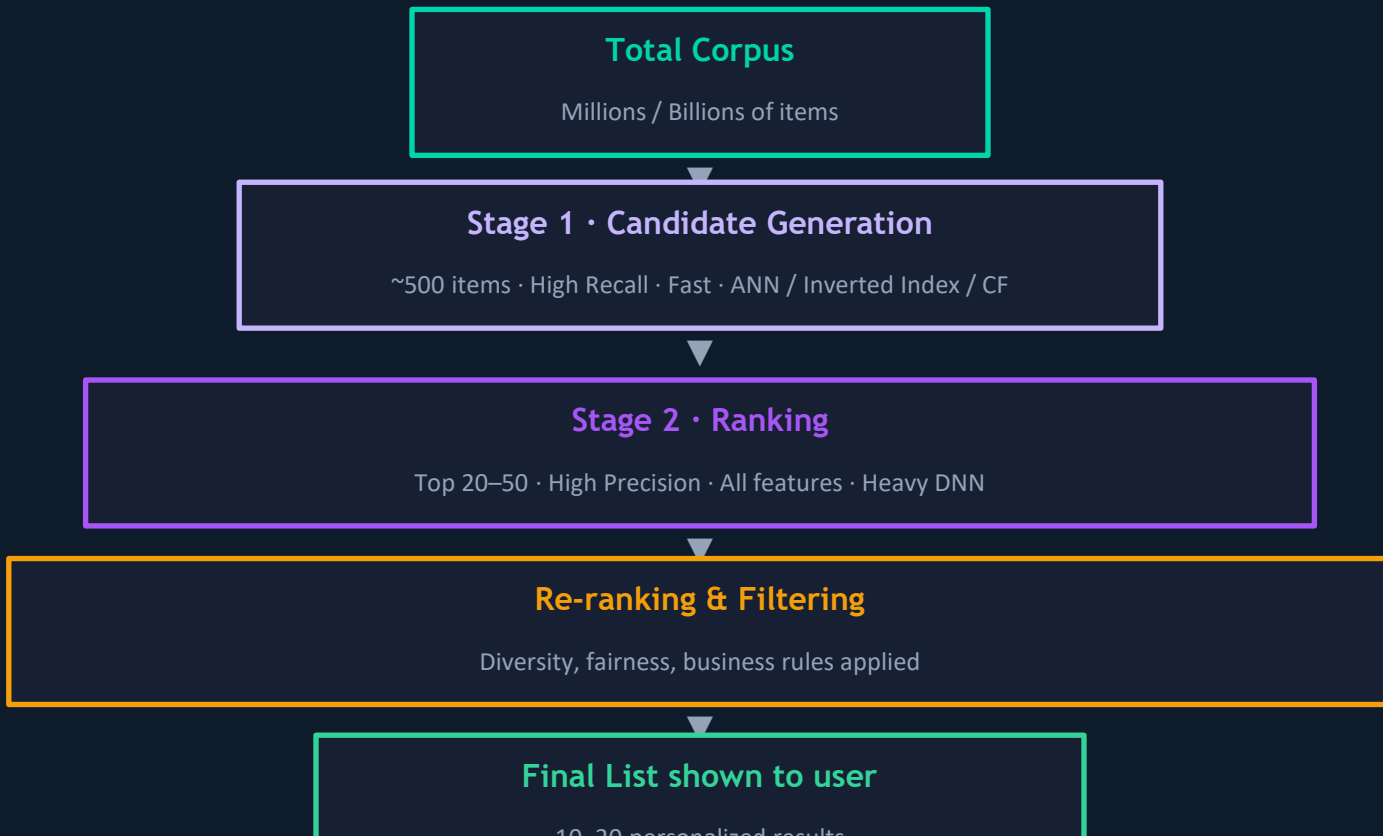
Session-based models. GRU4Rec uses recurrent networks;
BERT4Rec applies Transformer self-attention to click sequences.

Graph Neural Networks (GNN)

PinSage (Pinterest): 3B nodes, 18B edges. Random-walk aggregation of image + text features from neighbors.

Production Pipeline: Funnel Architecture

Real systems use a multi-stage funnel: fast broad retrieval → slow precise ranking.



Evaluation Metrics

RMSE / MAE

Use case: Explicit ratings

Measures **distance between predicted and actual rating**.
Penalizes large errors. Standard for Netflix Prize.

Precision@k

Use case: Top-N recommendation

Of the top-k items shown, how many did **the user actually like**?
Measures accuracy of the short list.

Recall@k

Use case: Top-N recommendation

Of all items the user likes, how many **did the system find**?
Balances with Precision.

NDCG

Use case: Ranked lists

Normalized Discounted Cumulative Gain. A hit at Rank #1 is worth more than a hit at Rank #10.

MRR

Use case: Single-answer queries

Mean Reciprocal Rank. How high is the first relevant result?
Perfect for voice assistants (Siri, Alexa).

A/B Testing (Online)

Use case: Gold standard

Real user behavior — CTR, watch time, revenue, retention.
Offline metrics are proxies; A/B is truth.

Solving the Cold Start Problem

New User: no history to find neighbors. New Item: no ratings to recommend from.

Hybrid Approach

Fallback to Content-Based when interaction data is missing. Use genre, cast, and keywords to make initial guesses.

Onboarding (Active Learning)

Ask for preferences at sign-up. Force users to rate 3–5 items or pick genres. Netflix "Pick 3 movies you like."

Popularity Baseline

When in doubt, recommend what's popular globally or locally. Safe, high-probability bet for most new users.

Explore / Exploit (Bandits)

Show new items to a small fraction of users to gather data quickly, even at risk of a bad recommendation. Used by TikTok.

Fairness & Bias in RecSys

"Algorithms are not neutral — they amplify patterns found in training data."

Popularity Bias

The rich get richer. Top 1% of items get 90% of exposure. Niche creators starve; users miss unique content.

Position Bias

Users click Rank #1 just because it's first. Model learns a self-fulfilling prophecy. Fix with Inverse Propensity Scoring (IPS).

□ Filter Bubbles

Optimizing engagement leads to **echo chambers**. Users are isolated from opposing views. Risk of **political polarization**.

Fairness Solutions

Consumer fairness (serve all demographics equally) + Provider fairness (give all creators exposure). Diversity constraints & re-ranking.

Diversity & Serendipity

The Echo Chamber

High Accuracy, but Boring

1. Star Wars Ep 4

2. Star Wars Ep 5

3. Star Wars Ep 6

4. Star Wars Ep 1

"I already know I like Star Wars."

The Discovery Mix

Balanced, Engaging & Surprising

1. Star Wars Ep 7 

2. The Office (Comedy) 

3. Planet Earth (Doc) 

4. Chef's Table (Food) 

"Oh, I haven't seen that documentary!"

Serendipity: finding something relevant AND unexpected — the happy accident that keeps users engaged long-term.

Context-Aware & Session-Based

Context-Aware: Rating = $f(\text{User}, \text{Item}, \text{Context})$

Context signals: Time · Location · Device · Social (alone vs. group) · Weather

Monday 8 AM · Mobile · Commuting

→  News Briefs & Short Podcasts

Saturday 8 PM · Smart TV · Family

→  4K Movies & Documentaries

Session-Based: Anonymous Users (no long-term history)

Must predict intent from the current click sequence only.

GRU4Rec (2016)

Treats clicks as a time series. Hidden state evolves with each action to capture short-term interest.

BERT4Rec (2019)

Self-Attention over the full session at once. Models complex dependencies (Item A relates to Item C, skipping B).

Explore / Exploit: Bandit Strategies

Should the system recommend what it knows you like (Exploit) or learn your new tastes (Explore)?

Exploitation

Recommend items with the highest predicted rating. Safe, high immediate reward — but the user gets stuck in a preference bubble.

Exploration

Recommend items we are uncertain about. Risky — user might dislike it — but we learn new preferences and improve over time.

ϵ -Greedy

With probability ϵ (e.g. 10%), pick a random item; otherwise pick the best known item. Simple but inefficient — explores bad items as often as good ones.

UCB (Upper Confidence Bound)

Score = Mean + Uncertainty. "Optimism in the face of uncertainty." Always picks items that could be the best. Theoretically optimal but assumes fixed distributions.

Thompson Sampling

Maintain a probability distribution for each item's success rate. Sample from each and pick the highest. Industry standard — handles complex distributions naturally.

Industry Case Studies

Netflix

SVD Matrix Factorization won \$1M Netflix Prize (2009)

Artwork personalization: different thumbnail per user taste

Two-stage retrieval + heavy ranking pipeline

A/B test everything — offline metrics are just proxies

YouTube

Two-Tower model: Query Tower (user) + Candidate Tower (video)

Item vectors pre-computed and stored in FAISS vector DB

Optimizes watch time, not clicks — avoids clickbait

Handles 500+ hours of video uploaded every minute

Spotify

Discover Weekly: 30 unheard songs users will love

Collaborative Filtering on billions of user playlists

NLP on music blogs with Word2Vec for artist adjectives

Audio CNNs on spectrograms — solves new-song cold start

Summary: The RecSys Spectrum

Content-Based

TF-IDF · User Profiles

Recommends items similar to past likes using metadata (genre, cast, keywords). User-independent, transparent, but prone to filter bubbles.

Collaborative Filtering

k-NN · User-User · Item-Item

Leverages crowd interaction patterns — no metadata needed. Serendipitous but suffers from cold start and data sparsity.

Matrix Factorization

SVD · ALS · Embeddings

Learns latent factor vectors for users and items. Captures hidden preferences. Foundation of modern production recommenders.

Deep Learning

NCF · Two-Tower · Transformers

Handles non-linear relationships, sequences, and multimodal features (text, image, audio). Powers today's largest platforms.

Hybrid Systems

Context · Bandits · Re-ranking

Modern production systems combine all of the above. Context-awareness, fairness constraints, and online learning close the loop.

Lab: RecSys with Surprise Library

Python toolkit for building recommender systems — the "scikit-learn for RecSys".



100,000

Ratings (1–5)



943

Users



1,682

Movies



93.7%

Sparsity

Included Algorithms

SVD / SVD++

KNNBasic

NMF

SlopeOne

CoClustering

BaselineOnly

```
from surprise import SVD, Dataset
from surprise.model_selection import cross_validate

data = Dataset.load_builtin('ml-100k')
algo = SVD(n_factors=100, n_epochs=20)

results = cross_validate(algo, data,
```



Key Takeaway: SVD (Model-Based) consistently outperforms KNN (Memory-Based). Lower RMSE means latent factors capture patterns that simple neighbor averaging misses.



Thank You

Questions?



Required Reading:

Koren et al. (2009) — Matrix Factorization Techniques | Covington et al. (2016) — YouTube Two-Tower



Discussion:

Echo chambers · Addiction ethics · Cold start strategy · Privacy vs. utility